

Atividade Objetiva 4

Entrega 2 jun em 23:59**Pontos** 1**Perguntas** 5**Disponível** 10 fev em 0:00 - 2 jun em 23:59 4 meses**Limite de tempo** Nenhum**Tentativas permitidas** 2

Instruções

Importante:

Caso você esteja realizando a atividade através do aplicativo "Canvas Student", é necessário que você clique em "**FAZER O QUESTIONÁRIO**", no final da página.

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MANTIDO	Tentativa 2	4 minutos	1 de 1
MAIS RECENTE	Tentativa 2	4 minutos	1 de 1
	Tentativa 1	54.448 minutos	0 de 1

Pontuação desta tentativa: **1** de 1

Enviado 11 abr em 16:39

Esta tentativa levou 4 minutos.

Pergunta 1

0,2 / 0,2 pts

Leia o trecho a seguir:

Pelo menos um bloco catch ou um bloco finally deve se seguir imediatamente ao bloco try. Cada bloco catch especifica entre parênteses um parâmetro de exceção que identifica o tipo de exceção que a rotina de tratamento pode processar. Quando ocorrer uma exceção em um bloco try, o bloco catch que é executado é o primeiro cujo tipo corresponde ao tipo da exceção que ocorreu (isto é, o tipo no bloco catch corresponde exatamente ao tipo de exceção lançado ou é uma superclasse direta ou indireta dele). O nome do parâmetro de

exceção permite ao bloco catch interagir com um objeto de exceção capturado.

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 352. Adaptado.

Considerando as informações apresentadas, analise as afirmações a seguir:

I. Uma função deve tratar todas as exceções que podem ocorrer em seu corpo, caso contrário o programa poderá ter sua execução interrompida por um erro.

II. Podemos criar nossas próprias exceções criando classes que herdem de Exception, uma classe padrão do Java.

III. Utilizamos o bloco catch para lançar exceções. Ele detecta que um erro ocorreu, e dentro de seu corpo utilizamos "throw" para lançar a exceção correspondente.

IV. Ao criarmos uma função que chama outras funções que lançam exceções, nossa função deverá tratá-las ou lançá-las também.

Estão corretas apenas as afirmativas:

☐ I e III, apenas.

☐ I, II e III, apenas.

☐ II, III e IV, apenas.

☐ I e IV, apenas.

☒ II e IV, apenas.

Correto!

Alternativa correta.

O bloco “catch” serve para tratar exceções. Dentro do bloco “try” é colocado o código potencialmente problemático, e blocos “catch” são utilizados para detectar diferentes erros, e em seu corpo é realizado o tratamento adequado.

Uma exceção encontrada precisa ser tratada para não provocar o encerramento abrupto do programa. O tratamento pode ser no ponto onde ela ocorreu, através de blocos catch, ou pode ocorrer em funções “superiores”, através do lançamento da exceção pela função “inferior” através do “throw”.

O Java providencia diversas exceções prontas, e cada uma é uma classe diferente, todas derivadas de Exception, a qual podemos utilizar para criar nossas próprias exceções para comunicar erros específicos em nossos projetos.

Pergunta 2

0,2 / 0,2 pts

Analise a linha de código abaixo:

```
class A extends B implements C {
```

O que podemos afirmar sobre A, B e C?

☐

A, B e C são classes, mas C é classe abstrata e seus métodos deverão ser implementados por A.

☐

A e C são classes, B é uma interface. A possui todos os métodos e atributos de C e deverá implementar os métodos de B.

Correto!

A e B são classes, C é uma interface. A possui todos os métodos e atributos de B e deverá implementar os métodos de C.

Alternativa correta.

A expressão "class A" denota que A é uma classe. A palavra "extends" indica herança, portanto B também é uma classe e A herda de B, portanto A passa a ter todos os atributos e métodos de B. O contrário não é verdade porque A pode criar novos atributos e métodos. A palavra "implements" indica interface, portanto C é uma interface. Ao contrário da herança, implementar uma interface não representa receber código pronto, e sim a obrigação da classe de implementar os métodos previstos pela interface.



A e C são classes, B é uma interface. B possui todos os métodos e atributos de C e deverá implementar os métodos de B.



A e B são classes, C é uma interface. B possui todos os métodos e atributos de A e deverá implementar os métodos de C.

Pergunta 3

0,2 / 0,2 pts

Leia o trecho a seguir:

"Métodos abstratos não fornecem implementações. Uma classe que contém quaisquer métodos abstratos deve ser expressamente declarada abstract, mesmo que ela contenha alguns métodos concretos (não abstratos). Cada subclasse concreta de uma superclasse abstrata também deve fornecer implementações concretas de cada um dos métodos abstratos da superclasse."

Fonte: DEITEL, H.; DEITEL, P. **Java: como programar**. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 317.

Considerando as informações apresentadas, analise as afirmações abaixo:

I. Podemos instanciar objetos de classes abstratas desde que elas não possuam métodos abstratos.

II. Uma classe herdeira de uma classe abstrata também será abstrata.

III. Uma classe concreta herdeira de uma classe abstrata deverá implementar todos os métodos abstratos.

IV. O polimorfismo permite que um objeto seja tratado como pertencente a uma classe abstrata.

Estão corretas apenas as afirmações:

☐ II e IV, apenas.

☐ I, III e IV, apenas.

☐ I e III, apenas.

☐ I e II, apenas.

☒ III e IV, apenas.

Correto!

Alternativa correta.

Uma classe abstrata não poderá ser utilizada para instanciar objetos. Ela serve apenas para herança, e objetos deverão ser instanciados a partir das classes herdeiras. Portanto, a afirmação I está incorreta.

A afirmação II também está incorreta, pois se classes herdeiras de classes abstratas também fossem abstratas, não poderíamos as utilizar para instanciar objetos, o que resultaria em uma “árvore” inteira de classes sendo criadas e não podendo ser utilizadas.

A alternativa III está correta, pois uma classe abstrata pode conter métodos abstratos, que são métodos não implementados. Sua assinatura (nome, tipo e retorno) é prevista na classe base (abstrata) para garantir que todos os herdeiros terão esse método, mas cada uma deverá implementá-lo à sua maneira. Caso algum método siga sem implementação concreta, a classe herdeira também será abstrata.

A alternativa IV também está correta, pois se a classe “A” é abstrata e as classes B, C e D são concretas, podemos criar funções que lidam com objetos da classe “A”. Por mais que eles não existam, objetos da classe B, C e D poderão ser utilizados por conta do polimorfismo. Métodos previstos pela classe A existem em B, C e D, e nossa função irá funcionar.

Pergunta 4

0,2 / 0,2 pts

Leia o trecho a seguir:

Interfaces definem e padronizam como coisas, pessoas e sistemas podem interagir entre si. Por exemplo, os controles em um rádio servem como uma interface entre os usuários do rádio e os componentes internos do rádio. Os controles permitem que os usuários realizem somente uma série limitada de operações (por exemplo, mudar de estação, ajustar o volume, escolher entre AM e FM) e diferentes rádios podem implementar os controles de diferentes maneiras (por exemplo, uso de botões, sintonizadores, comandos de voz). A interface especifica quais operações um rádio deve permitir

que os usuários realizem, mas não especifica como essas operações são realizadas.

Fonte: DEITEL, H.; DEITEL, P. **Java: como programar**. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 331.

Considerando a analogia apresentada, assinale a alternativa correta.



Ao implementarmos uma interface em nossas classes, as diferentes classes herdarão seus métodos. Esses métodos serão iguais em todas as classes.

Correto!



Ao utilizarmos interfaces em nossos projetos, não especificamos na interface detalhes internos de funcionamento, apenas os pontos de contato entre o objeto e outras estruturas do código.

Alternativa correta.

Uma interface é uma garantia de que diferentes objetos terão uma mesma funcionalidade. Isso é feito especificando a assinatura dos métodos na interface, porém sem especificar a implementação: cada classe que implementa a interface assume o compromisso de fornecer uma implementação. Desta maneira, a funcionalidade pode ser muito diferente internamente em diferentes classes – o que é bastante diferente de uma herança, onde uma classe filha automaticamente recebe todo o código da classe mãe. No caso da interface, implementá-la implica automaticamente em precisar gerar mais código: nossa classe será obrigada a ter uma implementação própria para cada um dos métodos. Porém, podemos aplicar o princípio do polimorfismo: um trecho de código preparado para trabalhar com um objeto que implemente aquela interface poderá trabalhar com objetos de qualquer classe que a implemente, pois ele com certeza irá oferecer o conjunto de métodos previsto na interface.



Uma interface não é uma classe e não serve para realizar herança, mas para padronizar pontos de contato entre objetos e diferentes estruturas no código, como assinaturas de métodos. Por conta disso, não podemos realizar polimorfismo utilizando interfaces.



Uma interface é uma classe especial, pois ela permite que alguns de seus métodos não sejam implementados, sendo conhecidos como métodos abstratos. Consequentemente, ela não poderá ser utilizada para instanciar objetos, apenas para herança.



O uso de interfaces é uma forma de economia de código em nossos projetos, pois com elas podemos fazer objetos de diversas classes diferentes terem acesso aos mesmos métodos.

Pergunta 5

0,2 / 0,2 pts

Leia o trecho a seguir:

Uma interface Java descreve um conjunto de métodos que pode ser chamado em um objeto para instruí-lo, por exemplo, a realizar alguma tarefa ou retornar algumas informações. (...) Uma declaração de interface inicia com a palavra-chave interface e contém somente constantes e métodos abstract. Diferentemente das classes, todos os membros de interface devem ser public e as interfaces não podem especificar nenhum detalhe de implementação, como declarações de método concretas e variáveis de instância. Todos os métodos declarados em uma interface são implicitamente métodos public abstract.

Fonte: DEITEL, H.; DEITEL, P. **Java**: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017. p. 331.

Considerando as informações apresentadas, analise as asserções abaixo e a relação proposta entre elas:

I. Podemos utilizar interfaces para implementar polimorfismo em nossos objetos.

PORQUE

II. Todas as classes que implementam uma mesma interface irão herdar os mesmos métodos, podendo, portanto, ser tratados como objetos de uma mesma classe.

A respeito dessas asserções, assinale a opção correta:

☐

A asserção I e a asserção II são proposições verdadeiras, e a II é uma justificativa correta da I.

☐

A asserção I e a asserção II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

☐

A asserção I é uma proposição falsa, e a asserção II é uma proposição verdadeira.

☐

Ambas as asserções são proposições falsas.

Correto!

☒

A asserção I é uma proposição verdadeira, e a asserção II é uma proposição falsa.

Alternativa correta.

Nós podemos utilizar interfaces para implementar polimorfismo, porque a interface será uma garantia de que todos os objetos da classe que a implementa irão possuir um certo conjunto de métodos. Portanto, funções projetadas para trabalhar com objetos que possuem uma certa interface poderão trabalhar com todos os objetos que possuem essa interface, independentemente de qual seja sua classe. A asserção I é uma proposição verdadeira.

Porém, implementar uma interface não significa herança. A classe que implementa uma interface não recebe nenhum tipo de código pronto. O polimorfismo pela interface surge da obrigação que a classe assume de fornecer alguma implementação para todos os métodos propostos pela interface. Portanto, a asserção II é uma proposição falsa.

Pontuação do teste: **1** de 1